YUVIZ · SERVICES/KNOWLEDGE

RAG Pipeline Redesign

How ingestion and retrieval work today, where they break, what replaces them, how each step fits the voice latency budget, and the order we build it in.

1 Today 2 Problems 3 New design 4 Chunking 5 Why it is better 6 Latency
7 Tool calls 8 Build plan 9 Decisions

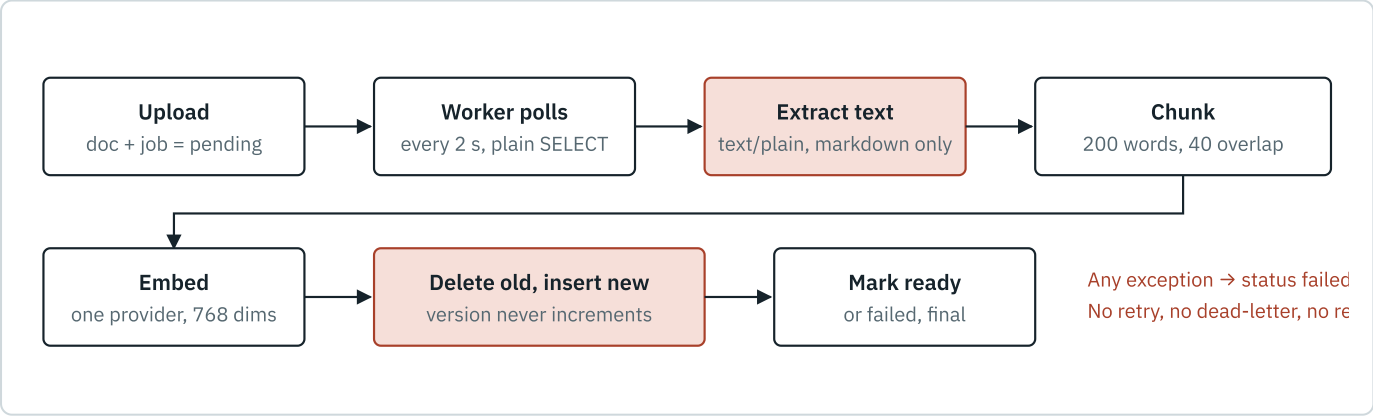
> Latency figures in this page are **budgets and estimates** derived from reading the code and typical provider and Postgres timings. None of them are measured yet. Phase 6 adds the benchmark that turns them into numbers.

1 · CURRENT SYSTEM

How it flows today

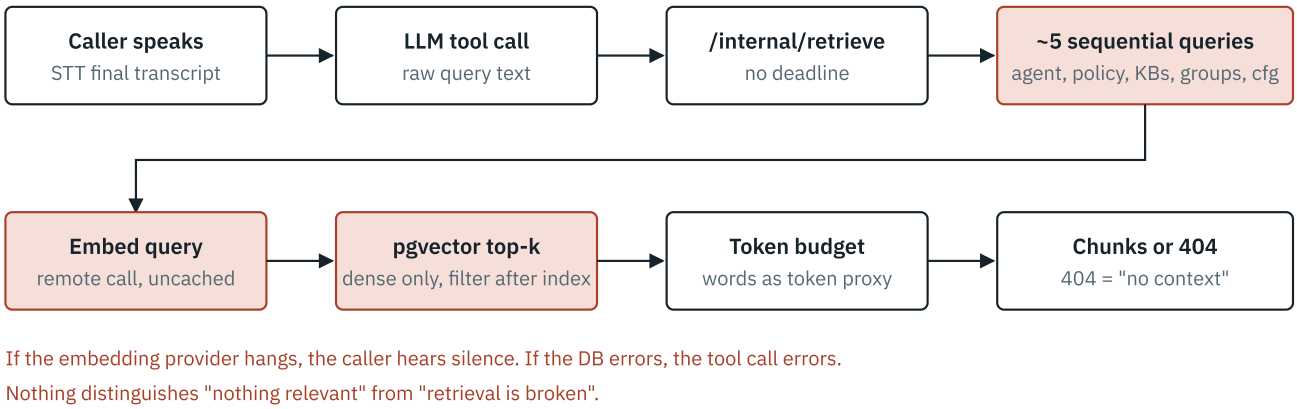
The pipeline works end to end and has solid tenant isolation (RLS on every table, a same-tenant trigger on `agent_knowledge_bases`, `tenant_conn` scoping in the worker). The weaknesses are in how it handles failure and how many steps sit on the call path.

Ingestion



Ingestion today. Red boxes are where a document is lost or silently wrong. Source: `services/knowledge/ingestion_worker.py`, `chunking.py`.

Retrieval during a call



Retrieval today. Source: `services/knowledge/retrieval.py` , `vector_repository.py` .

2 · WHY THAT IS NOT GOOD ENOUGH

What goes wrong, and where in the code

Each row is something visible in the current source. The right-hand column is what a tenant or caller experiences.

PROBLEM	WHERE	WHAT HAPPENS
Re-ingest never bumps `version` bug	`ingestion_worker.py` : `doc_row` is read after the status is set to `processing` , so the `status == "ready"` check is never true.	Versioned chunks never exist, so there is no way to swap atomically or roll back.
Job claim is a plain SELECT unsafe	`_claim_next_job` (its own comment says single worker only).	Two workers process the same job. A worker that crashes mid-job leaves it `running` forever.
No retry, no dead-letter loss	`attempts` is counted and never read. Any exception ends in `failed` .	A transient embedding timeout permanently fails the document. The tenant must re-upload.
Delete then insert chunks window	`DELETE FROM kb_chunks` followed by inserts.	While re-ingesting, retrieval can see fewer chunks, or none, for that document.
Only txt and markdown coverage	`SUPPORTED_CONTENT_TYPES` .	A PDF or DOCX is rejected outright. This is the main "fails for any document" gap.

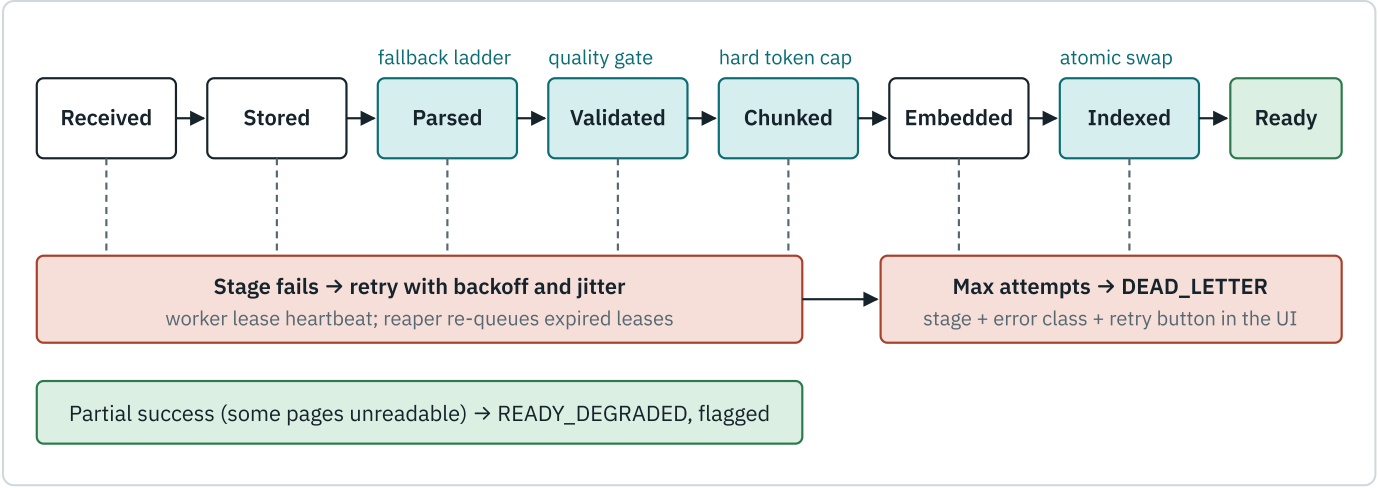
PROBLEM	WHERE	WHAT HAPPENS
Filtered HNSW under-returns `recall`	`vector_repository.py` : `WHERE kb_id = ANY(...)` then `LIMIT` , then a score filter in Python.	The index returns its nearest rows first and the filter removes them afterward, so the agent can get fewer than `top_k` , or nothing, even when a match exists.
About five sequential lookups per query `latency`	`retrieval.py` : agent id, policy, KB ids, KB groups, embedding config.	Fixed cost on every retrieval before any real work, and every one of them is a failure point.
No deadline, no fallback `silence`	The router and `retrieve()` . 404 means "no context".	A slow embedder stalls the turn. "Broken" and "nothing found" look the same to the agent.
Words as token count `accuracy`	`chunking.py` , `_approx_token_count` .	Hindi, Arabic and code tokenise very differently from whitespace words. Chunks can exceed the embedder limit and the budget is wrong.
Dense search only `quality`	`hybrid_search` and `rerank` exist in the policy and are not implemented.	Names, product codes and numbers, which callers say constantly, are the queries embeddings handle worst.

3 · TARGET DESIGN

The new flow

Two rules drive everything. **Ingestion is a durable state machine**: every document ends in a terminal state with a reason, and none is ever lost or stuck. **Retrieval is never allowed to break a call**: every stage has a deadline and a fallback, and the result says honestly whether it found nothing or failed.

Ingestion: durable, idempotent, every failure has a place to go



New ingestion. Teal stages are new or rebuilt. Every transition is appended to an events table, so "where is my document" always has an answer.

Parsing never says "unsupported"

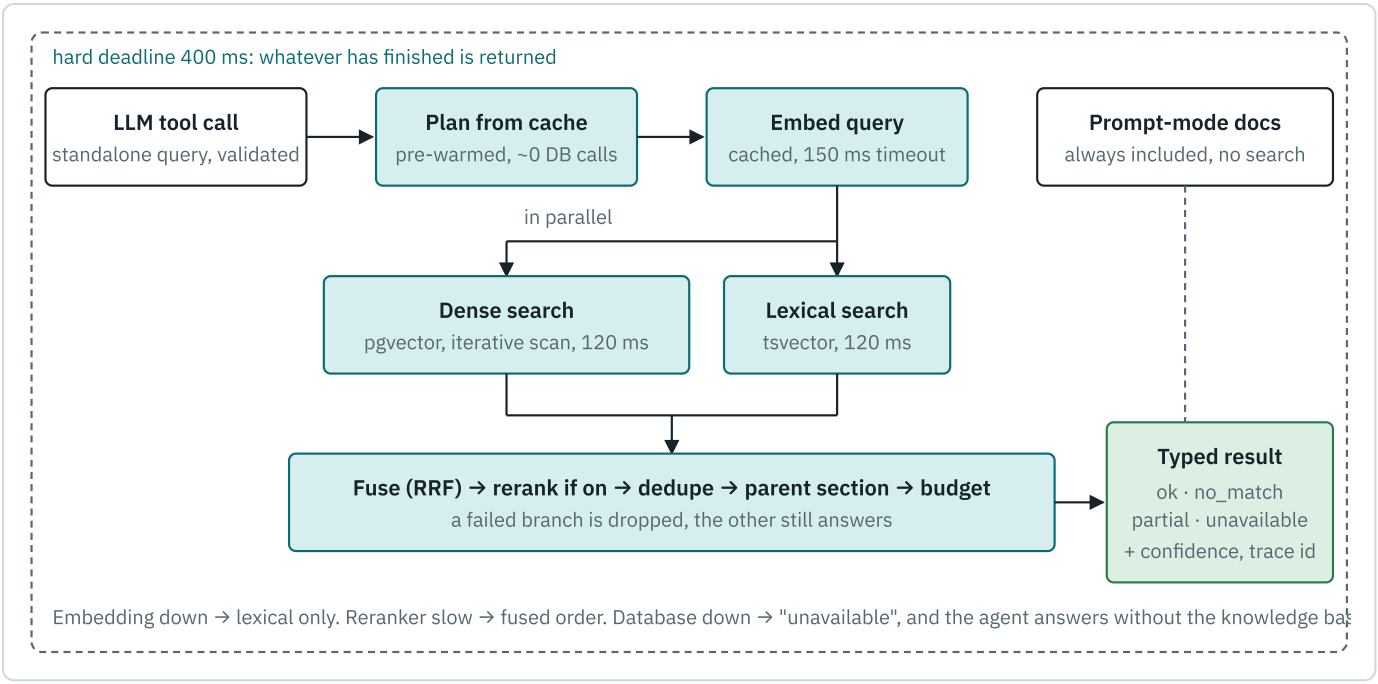
A router detects the real type from file bytes, then walks a fallback ladder per type. A quality gate checks the output (empty text ratio, garbage characters, characters per page, language confidence) before the document moves on.

- **PDF:** layout-aware text → plain text → OCR → vision model. A scanned PDF is detected by low characters per page.
- **DOCX, PPTX, XLSX:** native parser → convert to PDF → strip to text.
- **HTML:** readability extraction → raw text → headless render.
- **Corrupt, encrypted, oversized:** detected up front and rejected with a plain reason, not a stack trace.

Chunking and indexing that cannot half-finish

- Split on headings, keep tables whole, prepend a heading breadcrumb to each chunk.
- Hard token cap measured with the embedder's real tokenizer. Giant paragraphs fall back to sentence, then fixed windows.
- Small chunks are embedded, the parent section is returned for context.
- New-version chunks are written inactive, then one transaction flips `active_version`. Retrieval never sees a half-built document.
- A post-index probe queries the document's own first chunk and asserts it comes back, before `READY`.

Retrieval: bounded, parallel, honest



New retrieval. The dashed outer box is the deadline: nothing inside it is allowed to hang the call.

IF THIS FAILS	THE AGENT GETS	CALLER HEARS
Embedding provider slow or down	`partial` lexical-only results	A normal answer, slightly less semantic recall
Lexical search errors	`partial` dense-only results	A normal answer
Reranker over 120 ms	`partial` fused order	A normal answer
Nothing above the score threshold	`no_match` explicit	"I don't have that information." No invented answer
Database error or deadline hit	`unavailable` with a safe message	The agent says it cannot look that up and offers to transfer
Results found but scores close together	`ok` flagged `low_confidence`	The agent hedges or asks a clarifying question

4 · CHUNKING STRATEGY

How we choose how to chunk

We choose by measurement, not preference. The strategy named in section 3 (structure-aware, parent-child, heading breadcrumbs) is the **starting default**. It becomes the answer for a document type only if it wins a bake-off on that type's real questions.

What the use case forces

CONSTRAINT	EFFECT ON CHUNKING
The agent speaks one to three sentences	Small chunks retrieve precisely. Large ones dilute the embedding.
The LLM still needs context to answer without guessing	Match on a small chunk, return the enclosing section (parent-child).
Callers ask about names, prices and policy conditions	Tables and lists stay whole. The heading travels with every chunk.
Hindi, Arabic and mixed-language documents	Size is measured in the embedder's real tokens, never in words.
Tight latency budget (section 6)	No per-chunk LLM work at query time. All the cleverness happens at ingestion.
Unknown document types	The strategy must degrade gracefully when a document has no structure.

The candidates

STRATEGY	GOOD FOR	WEAKNESS	COST
Fixed window today: 200 words, 40 overlap	Anything, as a floor	Cuts mid-sentence and mid-table, drops headings	None
Recursive paragraph, then sentence, then window	Plain prose	Blind to headings and tables	None
Structure-aware	Policies, manuals, FAQs, HTML, DOCX	Needs a parser that keeps structure	None
Parent-child	Precise match plus enough context	More storage and one join	None at query time
Semantic split where similarity drops	Long unstructured text	Slow and unstable, often no better than recursive	Embeddings at ingest

STRATEGY	GOOD FOR	WEAKNESS	COST
Contextual enrichment LLM adds a situating line per chunk	Chunks that are ambiguous alone	Can add wrong context	LLM calls at ingest
Per-type special cases	FAQ sheets, tables, catalogues	More code paths	None

The selection process

STEP 1
Route by shape

Detect the document's shape and pick a strategy, with a safe default when detection finds nothing.

DOCUMENT SHAPE	STRATEGY	STARTING PARAMETERS
Has headings (policies, manuals, HTML, DOCX)	Structure-aware plus parent-child, breadcrumb prefix	Child 200–300 tokens, parent 600–1000, overlap 10–15%
Question and answer pairs	One pair per chunk	No overlap, question kept with its answer
Table or CSV	Row groups with the header row repeated in each	As many rows as fit under the token cap
Prose with no usable structure	Recursive	Child 200–300 tokens, overlap 10–15%
Everything else, or any splitter failure	Fixed window	Hard cap at the embedder's token limit

STEP 2
Size from the model

The hard cap is the embedding model's real token limit, counted with its own tokenizer. Starting sizes above are tuned in step 3, not guessed once and kept.

STEP 3
Bake-off

1. Build a **golden set** per KB: 30 to 100 real questions, each with the chunk or section that should answer it. Take them from call transcripts where possible.

2. Index the same documents once per candidate strategy.
3. Score recall@5, MRR, and the share of questions whose answer lands inside the returned context.
4. Pick the winner **per document type**, not one global winner.

A more complex strategy has to beat a simpler one by a clear margin to be kept. Semantic chunking and contextual enrichment most often fail this test, so they stay opt-in until the numbers justify them.

STEP 4
Stay changeable

Each document stores a `chunker_version`. Changing strategy is a background re-ingest that goes through the Phase 1 version swap, so it flips atomically and can be rolled back. A strategy change is never a risky migration.

What would change the default

- **Mostly short FAQs:** parent-child adds little, and one pair per chunk wins.
- **Answers that span two sections:** use a larger parent, or add neighbour-chunk expansion.
- **Poor recall on pronoun-heavy text:** this is where contextual enrichment pays for itself.
- **Your tenants' real document mix:** the routing table above is a hypothesis until we know what is actually uploaded (policy PDFs, FAQ sheets, catalogues, call scripts).

5 · BEFORE AND AFTER

Why the new design is better

CONCERN	TODAY	NEW DESIGN
Document lost on a transient error	Permanently `failed`	Retried with backoff, then dead-lettered with a reason and a retry button
Worker crashes mid-job	Job stuck in `running`	Lease expires, reaper re-queues, idempotent stages make the re-run safe
Many workers	Unsafe	`FOR UPDATE SKIP LOCKED` claim, per-tenant concurrency cap
Re-ingest	Delete then insert, version stuck	Build inactive version, flip one pointer in a transaction, clean up after

CONCERN	TODAY	NEW DESIGN
File types	txt, md	PDF, DOCX, HTML, XLSX, scans, with a fallback ladder and a quality gate
Chunk quality	Word windows	Structure-aware, real tokenizer, hard cap, parent section on return
Recall	Dense only, filtered after the index	Dense plus lexical fused, iterative scan so filters do not starve results
Failure behaviour	404 or exception	Typed status with a fallback per stage and a deadline on the whole path
Debuggability	Logs only	Ingestion events table and a per-retrieval trace with candidates and scores per stage
Regressions	Found by callers	Golden-set evals and a chaos corpus run in CI, a drop blocks the rollout

6 · LATENCY

Where every millisecond goes

Retrieval sits on the voice path, so it has a budget. The honest summary: the typical case gets modestly faster, and the real gain is that the **worst case is capped**. Today a slow provider can stall a turn indefinitely. In the new design the call path cannot exceed the deadline.

Typical retrieval, step by step

Estimated milliseconds on the call path. Bar scale is 0 to 200 ms.

Today	
5 sequential DB lookups	15
Embed query	60
Vector search	30
Budget and respond	1
Total	**~106**
New design, rerank off (default)	
Plan from cache	0.1

Embed query (miss)	60
Dense ‖ lexical, parallel	30
Fuse, dedupe, parent, budget	3
Total, embed cache miss	**~93**
Total, embed cache hit	**~34**
Optional rerank, when on	+80

Why each step costs what it does, and what removes the cost

STEP	BUDGET	MECHANISM
Plan resolution agent, policy, KBs, embedding config	≈0 ms	Resolved once per agent and held in memory, keyed by `config_version` so any change invalidates it. Pre-warmed when the call starts, so even the first question pays nothing. Today this is about five sequential round trips on every query.
Query rewrite for follow-ups	0 ms	The agent LLM already writes the tool argument, so the tool description instructs it to send a **standalone** query ("refund policy for the premium plan", not "what about the premium one"). No extra model call on the hot path.
Query embedding	1 ms hit 40–80 ms miss	Cache keyed by tenant, embedding config and normalised text. Callers repeat questions, and the agent often retries a phrasing. A 150 ms timeout, then the lexical-only fallback.
Dense search	20–40 ms	HNSW with iterative scan so the KB filter does not under-return. Runs in parallel with lexical search, so the pair costs the slower of the two, not the sum. 120 ms timeout.
Lexical search	10–20 ms	GIN index on a `tsvector` column. Hidden behind the dense search because they run concurrently.
Fusion, dedupe, parent expansion	2–4 ms	In-process work on at most about 60 candidates. Parent sections come back in the same query as the chunks, with no second round trip.
Rerank `opt-in`	60–100 ms	Off by default. Enabled per agent through the existing `rerank` policy flag. Skipped when the top fused score is already decisive, and dropped if it exceeds 120 ms.

STEP	BUDGET	MECHANISM
Whole path	`400 ms hard`	A deadline wraps everything. Whatever has finished at 400 ms is returned. A circuit breaker per dependency opens after repeated failures, so a dead provider costs nothing after the first few calls.

What is kept off the call path entirely

- Parsing, OCR, chunking, embedding documents, contextual enrichment and re-embedding all run in the ingestion worker. A caller never waits on them.
- Answer-time cost is a function of the query only, not of document size or type.
- Retrieval is a tool the LLM chooses to call, so chit-chat turns pay zero. Prompt-mode documents are included without any search.
- If a tool runs past about 800 ms, the agent speaks a short filler line so the call never goes silent.

Targets we commit to and will measure

MEASURE	TARGET	HOW IT IS CHECKED
Retrieval p50	`< 100 ms`	Benchmark harness against a seeded KB at realistic size
Retrieval p95, rerank off	`< 250 ms`	Same harness plus production histogram
Retrieval p99	`≤ 400 ms`	Fault injection: slow embedder, slow DB, dead reranker. The deadline must hold
Retrieval error surfaced to the caller	`< 0.1%`	Every error becomes `partial` or `unavailable`, counted separately
Document reaches a terminal state	`99.9% in 10 min`	Chaos corpus in CI and a stuck-job alert
Stale knowledge after an edit	`no half-states`	Atomic version swap test: retrieval during re-ingest never sees a gap

7 · TOOL CALLS

Making tool calls not fail

Retrieval is one tool among several, and the same envelope wraps all of them. The goal is that a tool problem becomes a sentence the agent can say, never a crash or a silent line.

```
{ "status": "ok | no_match | partial | unavailable | error",
  "data": { ... },
  "error": { "code": "...", "retryable": true, "user_safe_message": "..." },
  "latency_ms": 180,
  "trace_id": "..." }
```

Before the call

- Arguments validated against the tool's JSON schema. A bad argument returns a structured error the LLM can fix, with one repair attempt.
- `tenant_id`, `agent_id` and the allowed KB ids are injected by the executor. The model cannot name another tenant's data.
- A maximum number of tool calls per turn, and detection of the same call repeated.

During and after

- Per-tool timeout and a circuit breaker with a fallback: cached answer, degraded answer, then "I can't do that right now, can I transfer you?".
- Read tools retry once. Write tools use an idempotency key and are never retried blindly.
- Output validated, truncated to a token budget, and wrapped as data. Retrieved text is marked as untrusted so a prompt-injection inside an uploaded document is not followed.
- Every call logged with tenant, agent, redacted arguments, status and latency.

Not yet read: `services/toolexec`. Phase 2 starts by reading it and adapting this envelope to what already exists there, rather than adding a parallel mechanism.

8 · HOW WE BUILD IT

Build plan, in the order that removes risk first

Phases 1 and 2 give the "does not fail" guarantee. Phases 3 to 5 widen it to any document and cut the tail latency. Every phase ships behind tests, and every new tenant-owned table gets its RLS policy in `database/rls.sql` in the same change, or `tests/test_rls_coverage.py` fails.

PHASE 1	**Make ingestion safe to run and impossible to lose**

Queue and correctness

CHANGE — Claim with `FOR UPDATE SKIP LOCKED`. Add lease and heartbeat, retry with backoff, `max_attempts`, error class, dead-letter state, and an append-only `kb_ingestion_events` table. Fix the version bump by reading the document before the status flip. Write new chunks inactive and flip `active_version` in one transaction.

FILES — `ingestion_worker.py`, `database/knowledge_schema.sql`, `database/rls.sql`

EXIT WHEN — Killing a worker mid-job recovers the job. Two workers never take the same job. Re-ingest bumps the version and retrieval never sees a gap. A permanent error lands in dead-letter with a reason.

PHASE 2

Tool-call envelope

No tool problem reaches the caller as silence

CHANGE — Typed envelope, schema validation in and out, per-tool timeouts, circuit breakers, loop guard, executor-injected tenant scope. Split "no match" from "unavailable" in `/internal/retrieve` instead of using 404 for both.

FILES — `services/toolexec/`, `routers/retrieve.py`, the conversation service's tool caller

EXIT WHEN — Fault-injection tests (timeout, 500, malformed args, repeated call) each produce a defined status and a sayable message. A tenant id in tool arguments is ignored.

PHASE 3

Parser ladder

Any document ends in a defined state

CHANGE — Type detection from bytes, per-type fallback ladders, OCR path, size and page limits, sandboxed parser process, quality gate with a score stored on the document.

FILES — new `services/knowledge/parsers/`, `ingestion_worker.py`

EXIT WHEN — The chaos corpus (corrupt, encrypted, scanned, huge, RTL, mixed encoding, empty, zip bomb) ends every file in `READY`, `READY_DEGRADED` or `DEAD_LETTER` with a reason. None stays in `processing`.

PHASE 4

Chunking

Chunks that fit the model and carry their context

CHANGE	Structure-aware splitter with heading breadcrumbs, tables kept whole, real tokenizer, hard cap with sentence and fixed-window fallback, parent-child storage, `embedding_model` and `dims` on each chunk, provider failover.
FILES	`chunking.py` , `embedding_manager.py` , schema
EXIT WHEN	No chunk exceeds the embedder limit in any test document, including Hindi and Arabic. A failed embedding batch retries on its own without redoing the document. The strategy per document type is chosen by the section 4 bake-off on a first golden set (the full harness lands in Phase 6), not left at the default.

PHASE 5
Retrieval hot path

Fast in the typical case, bounded in the worst

CHANGE	Per-agent resolved plan cache keyed by `config_version` and pre-warmed at call start. Query-embedding cache scoped by tenant. `tsvector` column and GIN index. Parallel dense and lexical search with RRF fusion. Iterative HNSW scan. Per-stage timeouts, circuit breakers, 400 ms deadline, the fallback ladder.
FILES	`retrieval.py` , `vector_repository.py` , `cache.py` , schema
EXIT WHEN	With the embedder stalled, the DB slowed and the reranker dead in turn, p99 stays at or under 400 ms and the agent always gets a typed status. A cache entry for tenant A is never served to tenant B.

PHASE 6
Measure and guard

Turn the budgets into numbers and keep them there

CHANGE	Benchmark harness for the latency targets, golden question sets per KB (recall at k, MRR), chaos corpus wired into CI, `retrieval_logs` with a per-stage trace, dashboards and alerts for queue depth, dead-letter size, fallback rate and no-match rate.
EXIT WHEN	The section 6 target table is filled with measured values. A drop in recall or a slower p95 fails the build.

PHASE 7
Precision and scale

Optional quality gains once the base is solid

CHANGE	Hosted reranker behind the per-agent flag, contextual enrichment that fails open, re-embed jobs with a shadow index

	and atomic swap, connectors and scheduled re-sync.
EXIT WHEN	Golden-set metrics improve with rerank on, and p95 with rerank on stays inside the 400 ms deadline.

Schema additions

```
kb_documents        + content_hash, parser_used, quality_score, active_version,
                      status += READY_DEGRADED | DEAD_LETTER
kb_chunks           + parent_id, embedding_model, dims, is_active, tsv tsvector (GIN)
kb_ingestion_jobs   + stage, lease_expires_at, max_attempts, next_attempt_at, error_cl
kb_ingestion_events   new, append-only: job_id, stage, status, duration_ms, detail
retrieval_logs        new, tenant-scoped: trace_id, agent_id, query, status,
                      candidates, latency per stage
```

9 · DECISIONS

Three calls that are yours to make

OCR and vision parsing

A hosted service gives the best quality, and customer documents leave your infrastructure. Self-hosted keeps them private and is weaker on tables. This is a security decision as much as a quality one, so it needs your answer before Phase 3.

Job orchestration

Keep the Postgres queue, which needs no new infrastructure and is enough at this scale, or adopt a workflow engine such as Temporal. Recommendation: stay on Postgres, with the lease, retry and event table described above.

Reranker

A hosted reranker adds about 80 ms and a per-call cost for a precision gain. Recommendation: build it behind the existing per-agent `rerank` flag, default off, and decide from the golden-set results.

Grounded in a read of `database/knowledge_schema.sql` and `services/knowledge/` (ingestion worker, chunking, vector repository, retrieval). `services/toolexec` and the

conversation service's tool path have not been read yet. All latency figures are estimates until Phase 6.